

Rapport professionnel — Accompagnement du choix et intégration d'un service d'intelligence artificielle

Titre professionnel Développeur en Intelligence Artificielle — Bloc 2,
Épreuve E2

Veille, benchmark et paramétrage d'un service de prédiction de
tendance crypto

Projet support : Classification de Tendance Crypto

Certification : RNCP37827 — Développeur en Intelligence Artificielle

Compétences visées : C6, C7, C8

Dépôt : [crypto-certification \(GitHub\)](#)

Sommaire

1. Contexte du projet
2. Organiser une veille technique ciblée (C6)
3. Identifier et recommander un service d'IA à partir du besoin (C7)
4. Paramétrer le service retenu (C8)
5. Conclusion
6. Sources consultées

1. Contexte du projet

1.1 Le besoin à l'origine du projet

Le projet « Classification de Tendance Crypto » vise à construire une plateforme d'aide à la décision pour les investisseurs en cryptomonnaies. L'objectif fonctionnel est de prédire, pour chaque pas de temps (heure ou jour), si le prix du Bitcoin va monter (UP), descendre (DOWN) ou rester stable (STABLE), à partir de données de marché historiques et de signaux complémentaires.

La couche données (Bloc 1) collecte et expose via API REST les cours OHLCV et les paires de trading. Un scraper Scrapy (hébergé dans Bloc1_data, mais piloté depuis Bloc2) alimente séparément un fichier local d'actualités Bitcoin, consulté directement par le POC Streamlit sans passer par l'API ni par le modèle de prédiction. La question qui ouvre ce rapport est celle de la couche intelligence artificielle : quel service d'IA préexistant utiliser pour produire cette prédiction de tendance ?

Ce rapport couvre le choix et le paramétrage d'un service d'IA (C6-C8) ; l'exposition du modèle, son intégration dans l'application Django, le monitoring et les tests sont couverts par les rapports E3 et E4.

1.2 Première hypothèse de travail et ses limites

En tant qu'alternant ingénieur IA, je manipule quotidiennement des appels API LLM (extraction, agents, résumé). Ma première hypothèse de travail a donc naturellement été de tester un LLM généraliste (Anthropic Claude) en zero-shot comme baseline rapide pour la prédiction de tendance (une démarche que la littérature récente en finance computationnelle utilise elle-même comme point de comparaison initial avant de proposer des architectures spécialisées, cf. §3.3), en lui envoyant les données de marché des 30 derniers jours (Fear & Greed Index + cours BTC en USD) et en lui demandant de prédire la tendance des prochaines 24 heures. Cette approche est implémentée dans l'application Streamlit Bloc2_veille/app_benchmark.py.

Cette première tentative fonctionne au sens où elle produit une réponse structurée, mais révèle quatre limites structurelles :

Limite	Constat	Impact
Coût en tokens	~800-1 200 tokens input par requête (tableau 30 jours)	~0,003 \$/requête, non soutenable à volume élevé
Non-reproductibilité	Réponses variables même à	Impossible de garantir la

	temperature=0	cohérence des prédictions
Pertinence des données	LLM non entraîné sur données financières récentes	Précision faible sur une tâche quantitative
Empreinte carbone	Inférence GPU cloud à chaque requête, sans chiffre officiel publié par l'éditeur	Non quantifiable précisément, argument qualitatif détaillé en §3.5

Ce constat a motivé une veille formalisée sur deux axes : les indicateurs de sentiment et sources de données pouvant enrichir la prédiction, et le suivi des services IA eux-mêmes (nouveau, coûts réels, outils de suivi comme LiteLLM), pour garder le LLM sous contrôle en tant qu'outil complémentaire d'analyse qualitative, plutôt que comme boîte noire coûteuse. Le choix du service de prédiction lui-même (LLM généraliste vs modèle ML spécialisé) fait l'objet d'un benchmark séparé, présenté plus loin dans ce rapport.

2. Organiser une veille technique ciblée (C6)

2.1 Thématique et planification

La thématique retenue est : « Services d'IA pour la prédiction de tendance crypto : LLM généraliste vs modèle ML supervisé, évalués sur le coût, les tokens, la reproductibilité et l'éco-responsabilité ». Elle est directement mobilisée dans la mise en situation : il s'agit d'un choix d'outil pour un besoin concret du projet, pas d'un sujet générique.

2.2 Première veille : indicateurs de sentiment et sources de données

Avant de choisir un service d'IA, il fallait identifier des signaux de marché complémentaires aux cours OHLCV bruts. La veille a porté sur les indicateurs de sentiment crypto disponibles via API publique.

Source	Type	Données	Fiabilité
Alternative.me	API REST gratuite	Fear & Greed Index (score 0–100, classification)	Éditeur identifié, API stable depuis 2018, données quotidiennes
CoinGecko	API REST gratuite	Cours BTC/USD historiques 30 jours	Acteur reconnu, documentation OpenAPI, rate-limit documenté
CoinTelegraph	Scraping Scrapy (Bloc1)	Actualités Bitcoin (titre, date, catégorie)	Source média reconnue, scraping

			respectueux (robots.txt, délai 2s)
--	--	--	---------------------------------------

Le Fear & Greed Index a été retenu comme indicateur principal de veille car il mesure le sentiment du marché crypto sur une échelle 0 (peur extrême) à 100 (avidité extrême), publié quotidiennement par Alternative.me. Il est utilisé en entrée du prompt envoyé au LLM (parametrage.py, app_benchmark.py) ; il n'est en revanche pas utilisé par le modèle ML retenu (XGBoost, Bloc3), dont les features sont entièrement dérivées des OHLCV (cf. §3.3).

2.3 Seconde veille : suivre les services IA, de l'annonce au coût réel

La comparaison chiffrée détaillée des services (précision, tarifs, contraintes) est traitée en tant que telle au §3 (benchmark C7) : Anthropic Claude, OpenAI GPT-4o et Mistral Large y sont comparés à partir de leurs pricing pages officielles. Cette seconde veille répond à une question complémentaire et opérationnelle, en deux temps : comment rester informé des nouveautés de ces services une fois le choix fait, et comment suivre ce qu'ils coûtent réellement à l'usage ?

A. Rester informé des nouveautés (outils d'agrégation) :

- **Flux RSS des blogs éditeurs (signal direct)** : OpenAI (openai.com/news/rss.xml) et Mistral AI (mistral.ai/news/rss) publient un flux RSS public de leurs annonces modèles/produits, vérifié fonctionnel et intégré dans un onglet dédié de app_benchmark.py.

- **Flux Atom des releases GitHub (proxy imparfait pour Anthropic)** : Anthropic ne publie aucun flux RSS public sur son site. À défaut, le flux Atom natif des releases du SDK anthropic-sdk-python sert de signal de substitution : une nouvelle version du SDK accompagne souvent, mais pas systématiquement, un nouveau modèle. Ce n'est pas équivalent à une annonce directe, et le rapport le documente comme tel plutôt que de les confondre.

- **Flux Atom des releases GitHub (signal direct pour XGBoost)** : XGBoost est le modèle/la bibliothèque elle-même : sa release GitHub est un signal direct, sans ambiguïté.

B. Suivre le coût réel à l'usage (déclencheur : limite de monitoring identifiée en §4.2.3, le suivi manuel via la console Anthropic ne permettant ni traçabilité automatisée par clé/projet, ni comparaison multi-provider). Cette question fait l'objet d'une pratique professionnelle personnelle plutôt que d'une découverte en cours de projet : j'utilise LiteLLM depuis un an en alternance pour ce besoin exact.

LiteLLM est une bibliothèque et un proxy open-source qui expose une interface unifiée (format OpenAI) vers plus de 100 fournisseurs de LLM. Son mode proxy (litellm[proxy]) ajoute des clés API virtuelles avec suivi de dépense et budget par clé, adossés à une base PostgreSQL, et une UI d'administration. Comparé au suivi manuel (console fournisseur, un écran par provider, pas de budget configurable par clé), LiteLLM centralise le suivi token/coût quel que soit le fournisseur derrière, ce qui est pertinent ici puisque la production réelle et cette certification n'utilisent pas le même fournisseur. Le déploiement et le paramétrage effectifs de ce proxy sont détaillés en C8, §4.2.5.

2.4 Veille réglementaire

La veille réglementaire complète la veille technique et couvre deux cadres mobilisés dans le projet :

- **RGPD** : Registre des traitements documenté (docs/rgpd_registre_traitements.md) : les données OHLCV sont publiques ; seuls les comptes utilisateurs constituent des données personnelles.
- **MiCA (Markets in Crypto-Assets)** : Régulation UE applicable aux services crypto, pertinente car la plateforme affiche des prédictions de marché ; un disclaimer « aide à la décision, pas conseil financier » a été ajouté au pied de page de l'application (Bloc4_app/templates/base.html).
- **AI Act (UE)** : Obligations de transparence pour les systèmes d'IA à impact limité : le même disclaimer précise qu'une prédiction est produite par un modèle automatisé, et non par un conseiller humain.

2.5 Outils et modalités de partage

Outil	Usage	Justification
Script Python (parametrage.py)	Collecte automatisée Fear & Greed	Gratuit, reproductible, versionné sur Git
Streamlit (app_benchmark.py)	Visualisation et POC LLM	Interface accessible, déploiement local sans coût
GitHub Issues	Suivi des tâches (pas de board Kanban actif à ce jour)	Traçabilité, versionné avec le code, partage avec le formateur
Markdown (docs/)	Synthèses formalisées	Format texte structuré, consultable par lecteur d'écran

3. Identifier et recommander un service d'IA à partir du besoin (C7)

3.1 Reformulation du besoin

Besoin exprimé : disposer d'un outil capable de prédire la tendance du Bitcoin à court terme (24h) pour alimenter un dashboard d'aide à la décision destiné aux utilisateurs de la plateforme.

Reformulation technique : à partir de données de marché structurées (OHLCV historiques, et Fear & Greed Index pour le POC LLM), produire une classification ternaire (UP / STABLE / DOWN) avec un niveau de confiance, en respectant les contraintes suivantes :

Contrainte	Détail
Latence	< 5 secondes par prédiction
Budget	Limité : projet étudiant / certification, pas de budget SaaS récurrent
Reproductibilité	Prédictions identiques pour des données identiques
Éco-responsabilité	Minimiser l'empreinte carbone par inférence
Explicabilité	Capacité à justifier la prédiction (feature importance ou raisonnement textuel)

3.2 Services étudiés et non étudiés

Services étudiés avec test ou POC effectif :

- **Anthropic Claude Sonnet 4.6** : POC Streamlit, prédiction à partir de 30 jours de données (Fear & Greed + cours BTC).
- **XGBoost custom (Bloc3)** : Pipeline ML entraîné sur features OHLCV + indicateurs techniques (RSI, MACD, Bollinger, etc.).

Services écartés (raisons explicites, sans test direct) :

Service écarté	Raison
OpenAI GPT-4o	Pricing officiel consulté, pas de test direct : coût prohibitif (~0,01 \$/requête), pas de crédits gratuits pour un projet étudiant
Mistral Large	Documentation et tarification consultées, pas de test direct : crédits gratuits limités ; performance sur données tabulaires jugée a

	priori limitée par analogie avec la littérature citée en §3.3, sans benchmark Mistral spécifique recoupé
Services ML SaaS (AWS SageMaker, Google Vertex AI)	Coût d'infrastructure disproportionné pour un projet solo ; préférence pour un modèle auto-hébergé

3.3 Comparaison détaillée — LLM vs ML Custom

Critère	LLM (Claude Sonnet 4.6)	ML Custom (XGBoost Bloc3)
Précision	Faible : non entraîné sur données financières récentes	Moyenne à bonne : entraîné sur features OHLCV spécifiques
Reproductibilité	Faible : réponses non déterministes	Élevée : modèle déterministe à paramètres fixés
Latence	2-5 s (appel réseau + inférence cloud)	< 0,5 s (inférence CPU locale)
Coût par requête	~0,003 \$ (input + output tokens)	0 \$ (auto-hébergé, pas de coût marginal)
Éco-responsabilité	Forte empreinte : datacenter GPU	Faible : modèle léger, inférence CPU
Données d'entraînement	Corpus généraliste (coupure de connaissance)	OHLCV crypto récentes, mises à jour par cron
Explicabilité	Raisonnement textuel (non vérifiable)	Feature importance quantifiable (gain XGBoost)
Maintenance	Aucune : service géré par Anthropic	Ré-entraînement périodique (pipeline MLOps)

La ligne « Précision » s'appuie sur la littérature académique récente consacrée à l'usage de LLM comme outil de prédiction financière, et pas seulement sur le constat empirique du POC : Crisostomo & Mykhalyuk (2026) montrent que des LLM généralistes utilisés sans spécialisation ni supervision humaine forte souffrent d'erreurs de raisonnement récurrentes sur des tâches d'investissement ; Wang et al. (2024, StockTime) constatent que les LLM financiers non spécialisés « négligent les caractéristiques essentielles des séries temporelles », d'où la nécessité d'architectures dédiées pour obtenir une précision satisfaisante. Ces deux constats convergent avec le choix retenu ici : XGBoost, entraîné spécifiquement sur les séries temporelles OHLCV, plutôt qu'un LLM généraliste utilisé tel quel.

3.4 Adéquation par ensemble fonctionnel

Fonction	LLM (Claude)	ML Custom (XGBoost)
----------	--------------	---------------------

Prédiction quantitative de tendance	Non adapté	Conçu pour : couvrir le besoin principal
Analyse qualitative de marché	Pertinent	Non prévu
Synthèse d'actualités	Pertinent	Non prévu
Reproductibilité des résultats	Non garanti	Garanti
Explication du raisonnement	Textuel, non vérifiable	Feature importance quantifiable

3.5 Démarche éco-responsable

Ni Anthropic ni, plus largement, aucun fournisseur de LLM cloud ne publie de chiffre officiel de consommation énergétique ou d'empreinte carbone par requête : c'est une limite générale du secteur, pas propre à ce projet, et je ne dispose donc pas d'un chiffre sourcé à comparer. Les ordres de grandeur qui circulent dans la littérature généraliste sur l'inférence LLM cloud vs. un modèle CPU local n'ont pas été retenus ici faute d'avoir pu les recouper avec au moins deux sources indépendantes fiables.

Le raisonnement qualitatif que je peux tenir, sur la base des informations disponibles, est le suivant : pour la tâche de prédiction elle-même, XGBoost sollicite structurellement moins de calcul distant qu'un LLM cloud : inférence CPU locale légère (quelques millisecondes) contre un appel réseau vers une inférence GPU cloud à chaque prédiction, et un ré-entraînement limité à quelques minutes CPU par cycle. C'est un argument de sobriété par conception, pas un résultat mesuré.

3.6 Contraintes techniques et pré-requis

LLM (Anthropic Claude) :

- **Clé API** : ANTHROPIC_API_KEY nécessaire dans tous les cas, mais pas forcément détenue par le même composant : côté proxy LiteLLM quand il est configuré (le POC n'a alors besoin que d'une clé virtuelle émise par le proxy) ; directement côté POC sinon (cf. §2.3, §4.2.5).
- **Dépendances** : SDK Python anthropic, streamlit, pandas, requests
- **Connectivité** : HTTPS sortant obligatoire vers api.anthropic.com
- **Rate limit** : 1 000 req/min sur le tier gratuit

ML Custom (XGBoost Bloc3) :

- **Infrastructure** : Docker Compose (pipeline + API + MLflow)
- **Données** : Pipeline Bloc1 fonctionnel (API OHLCV alimentant les features)
- **Ré-entraînement** : Cron horaire et journalier (entrypoint.sh)
- **Dépendances** : scikit-learn, xgboost, pandas, pandas-ta, MLflow

3.7 Conclusions du benchmark

Résultats réels du backtest XGBoost (granularité journalière, période de test 2025-01-01 → 2025-05-31, métriques trackées dans MLflow après exécution du pipeline) :

Paire	Accuracy	F1 macro	Direction accuracy
BTC-USD	0,4133	0,3778	0,3889
BTC-USDT	0,4533	0,4292	0,4184

Ces résultats sont modestes mais réels et reproductibles : une accuracy de 41 à 45 % sur un problème à 3 classes (le hasard pur donnerait environ 33 % sur des classes équilibrées), cohérente avec la difficulté connue de prédire un mouvement de prix crypto à courte échéance. Le F1 par classe (disponible dans MLflow) montre en particulier que le modèle est plus fiable sur la détection de baisse/hausse marquée que sur la classe STABLE, un point identifié pour une itération future (ajustement du seuil de classification ou rééquilibrage des classes).

Le modèle XGBoost est évalué avec un protocole de backtest formel (fenêtres de test glissantes, métriques accuracy/f1_macro/direction_accuracy trackées dans MLflow sur la période 2025-01-01 à 2025-05-31). Le LLM, lui, n'a volontairement pas été soumis au même protocole de backtest systématique : chaque prédiction coûte un appel API payant, et la littérature récente (Crisostomo & Mykhalyuk 2026 ; Wang et al. 2024) montre que des LLM généralistes en zero-shot obtiennent une précision proche du hasard sur ce type de tâche : un backtest formel aurait probablement confirmé ce point sans justifier son coût. C'est précisément cette asymétrie de rigueur, mesurable pour l'un et coûteuse à mesurer pour l'autre sans gain attendu, qui motive le choix de XGBoost comme service de production.

Service retenu pour la prédiction de tendance : XGBoost (modèle custom Bloc3). Il est déterministe, reproductible, sans coût marginal, et sollicite structurellement moins de calcul distant qu'un appel LLM cloud (cf. §3.5 pour l'argument qualitatif détaillé ; aucun chiffre officiel d'empreinte carbone n'est disponible pour un ratio précis). Ce choix n'a pas fait l'objet d'une comparaison interne empirique avec d'autres familles d'algorithmes (Random Forest, régression logistique sont déclarés dans la configuration mais n'ont jamais été entraînés ni évalués) : il s'appuie sur la

réputation établie de XGBoost sur données tabulaires structurées (référence de facto sur ce type de problème), sa capacité à capturer des interactions non linéaires entre indicateurs techniques contrairement à un modèle linéaire, et son support natif de l'importance des features (explicabilité, cf. §3.3). Une comparaison empirique avec d'autres familles de modèles reste un axe pour une prochaine itération.

Le LLM (Anthropic Claude) reste pertinent comme outil complémentaire pour : la synthèse qualitative de conditions de marché, l'explication textuelle des tendances à destination d'utilisateurs non techniques, et l'enrichissement de l'interface (POC Streamlit). Aucun des deux services pris isolément ne couvre le besoin complet : l'un ne garantit pas la reproductibilité quantitative, l'autre ne produit pas d'analyse en langage naturel.

4. Paramétrer le service retenu (C8)

Le paramétrage couvre les deux composants de l'architecture retenue : le modèle ML XGBoost (service principal) et le LLM Claude (service complémentaire, POC démontrable).

4.1 Paramétrage du service principal — XGBoost (Bloc3)

4.1.1 Environnement d'exécution

Composant	Configuration
Image Docker pipeline	pipeline.Dockerfile : Python 3.11, cron horaire + journalier
Image Docker API ML	api.Dockerfile : FastAPI port 8002
Image Docker MLflow	mlflow.Dockerfile : UI monitoring port 5000
Orchestration	docker-compose.yml : volumes partagés ml_models, mlruns

4.1.2 Configuration du modèle

Les hyperparamètres et la sélection d'algorithme sont externalisés dans des fichiers YAML versionnés sur Git :

- **config/hour_models_config.yaml** : Modèle horaire par paire de trading (XGBClassifier par défaut).
- **config/day_models_config.yaml** : Modèle journalier par paire de trading.
- **config/ml_config.yaml** : Seuil de classification (0,5 %), fenêtres d'entraînement/test.
- **config/data_config.yaml** : Paramètres de récupération des OHLCV depuis l'API Bloc1.

Paramètre	Valeur	Justification
Algorithme	XGBClassifier	Réputation établie sur données tabulaires structurées (cf. §3.7) ; pas de comparaison interne empirique avec d'autres algorithmes
n_estimators	200	Nombre d'arbres du modèle : valeur par défaut XGBoost, non optimisée par recherche d'hyperparamètres ; compromis entre capacité et

		temps d'entraînement (quelques minutes CPU par cycle, cf. §3.5)
max_depth	6	Profondeur maximale de chaque arbre : valeur par défaut XGBoost, non optimisée ; limite le risque de sur-apprentissage sur un jeu de données de taille modeste
learning_rate	0,1	Poids de chaque arbre successif dans l'ensemble : valeur par défaut XGBoost, non optimisée ; compromis standard entre vitesse de convergence et stabilité
Classes	0=DOWN, 1=STABLE, 2=UP	Classification ternaire avec seuil 0,5 %
Features	RSI, MACD, Bollinger, SMA, EMA, lags, rendements	Indicateurs techniques standard en analyse crypto
Période entraînement BTCUSDT daily	2020-01-01 → 2024-12-31	5 ans de données pour robustesse
Période test	2025-01-01 → 2025-05-31	Validation out-of-sample

4.1.3 Pipeline et ré-entraînement

Le script `update_models_and_predictions.py` exécute le pipeline complet : récupération OHLCV via JWT Bloc1 → feature engineering → entraînement → évaluation → prédiction → envoi vers Bloc1 → sauvegarde pickle. Il est déclenché par cron (`entrypoint.sh`) : horaire à XX:05, journalier à 00:10.

4.1.4 Monitoring (MLflow) et accès

MLflow (port 5000) track les métriques `accuracy`, `f1_macro` et `direction_accuracy` à chaque exécution du pipeline. C'est le monitoring natif du service ML retenu, accessible via `http://localhost:5000` après `docker compose up mlflow-server`.

Côté accès : l'API de classification (`ml-api`) est protégée par authentification JWT (`Bloc3_ml/src/api/routes/classify.py::get_current_user`, `Bloc3_ml/src/api/utlis/deps.py`), cohérente avec le système d'authentification de l'API Bloc1. L'interface MLflow, elle, n'est aujourd'hui pas authentifiée (`mlflow.Dockerfile` ne configure aucun mécanisme d'accès) : c'est un point de vigilance identifié, acceptable en environnement de développement local mais à

corriger avant toute exposition au-delà de ce cadre (par exemple via un reverse-proxy avec authentification, ou la variable MLFLOW_TRACKING_TOKEN).

4.2 Paramétrage du service complémentaire — Claude (POC Streamlit)

4.2.1 Configuration de l'appel API

Paramètre	Valeur	Justification
Modèle	claude-sonnet-4-6	Modèle Sonnet actuel (remplace claude-sonnet-4-20250514 déprécié)
Temperature	0.0	Réduit au maximum l'échantillonnage aléatoire (choix du token le plus probable à chaque étape), sans garantir une reproductibilité stricte sur une API cloud (cf. §3.3, non-déterminisme lié au batching côté serveur)
Max tokens	500	Initialement 256, augmenté après un test réel où une réponse a utilisé 245 tokens (11 de marge seulement, risque de troncature du champ reason) : 500 laisse une marge confortable, vérifiée sur 3 appels consécutifs (246-275 tokens utilisés)
Format de réponse	Tool use (schéma JSON : prediction / confidence / reason)	Sortie structurée forcée par le SDK Anthropic (PREDICTION_TOOL) plutôt qu'un parsing de texte libre, plus robuste (cf. §4.2.3)

4.2.2 Construction du prompt

Le prompt envoie au LLM un tableau fusionné (Fear & Greed + cours BTC) sur 30 jours, trié du plus récent au plus ancien, et demande une prédiction en appelant l'outil submit_prediction (tool use natif Anthropic, cf. §4.2.3) plutôt qu'un format texte à respecter.

Exemple de prompt (extrait) :

« Tu es un analyste crypto. Voici les données des 30 derniers jours pour Bitcoin :

Fear & Greed Index + Cours BTC en USD. [tableau]. En te basant uniquement sur ces données, prédis la tendance du Bitcoin pour les prochaines 24h en appelant l'outil submit_prediction. » Le schéma de l'outil impose prediction (HAUSSE/BAISSE/STABLE), confidence (0-100) et reason (texte) en sortie structurée.

4.2.3 Mesures collectées à chaque appel

Métrique	Usage
input_tokens / output_tokens	Estimation du coût par requête (tarif Anthropic Sonnet 4.6)
latency_s	Comparaison avec latence ML local (< 0,5 s)
prediction / confidence / reason	Résultat structuré affiché dans Streamlit

Ce suivi applicatif est complété par deux niveaux de monitoring externes : la console Anthropic (console.anthropic.com), monitoring natif du service SaaS, consultable manuellement par fournisseur ; et le proxy LiteLLM (§4.2.5), qui automatise ce suivi par clé API et centralise plusieurs fournisseurs : c'est l'outil effectivement utilisé pour vérifier que le coût réel par requête reste conforme à l'estimation faite dans le benchmark (C7).

4.2.4 Gestion des accès et sécurité

- **Clé API** : ANTHROPIC_API_KEY lue depuis .env via python-dotenv, jamais committée (.gitignore).
- **Saisie fallback** : Si la clé n'est pas en .env, Streamlit propose un champ password pour saisie manuelle.
- **Gestion d'erreurs** : anthropic.APIError capturée et affichée dans l'interface sans crash de l'application.

4.2.5 Paramétrage du proxy LiteLLM (monitoring token/coût, veille C6)

Suite à la veille du §2.3, un proxy LiteLLM a été déployé et paramétré comme service Docker du projet (litellm-proxy, docker-compose.yml), plutôt que documenté comme simple piste théorique :

- **Image** : ghcr.io/berriai/litellm:v1.90.2 (version épinglée), backend Prisma/PostgreSQL dédié (base litellm sur l'instance Postgres du projet).
- **Configuration du modèle** : litellm_config.yaml déclare claude-sonnet-4-6 (anthropic/claude-sonnet-4-6, clé lue depuis ANTHROPIC_API_KEY) : un seul modèle pour ce POC, extensible aux autres fournisseurs du benchmark sans changer le code applicatif.

- **Authentication** : LITELLM_MASTER_KEY (accès admin du proxy) et LITELLM_SALT_KEY (chiffrement des identifiants stockés), générées aléatoirement et lues depuis .env, jamais committées.

- **Clé virtuelle applicative** : Générée via POST /key/generate (scope : modèle claude-sonnet-4-6, budget maximal configuré) ; c'est cette clé, et non la clé Anthropic brute, qui est utilisée par app_benchmark.py.

- **Intégration applicative** : call_anthropic() route désormais l'appel via le SDK anthropic standard, avec base_url pointant vers le proxy (endpoint passthrough /anthropic). Aucune dépendance supplémentaire ; repli automatique sur l'appel direct si le proxy n'est pas configuré.

Vérification en conditions réelles : deux appels successifs via le proxy ont été tracés correctement (dépense cumulée passée de 0,000117 \$ à 0,000561 \$), consultable via GET /spend/logs, l'UI d'administration du proxy et dans l'application elle-même.

Point de vigilance : le déploiement initial a rencontré un OOM-kill silencieux (mémoire Docker insuffisante pour le client Prisma), corrigé en libérant de la mémoire sur l'hôte, documenté tel quel plutôt que masqué.

4.3 Sources de données utilisées

Aucune source n'est réellement commune aux deux services : le POC LLM consomme Fear & Greed Index et cours BTC (ci-dessous) ; XGBoost consomme uniquement les OHLCV historiques via le pipeline Bloc3 (cf. §4.1). Les actualités Bitcoin scrapées (cf. §2.2) ne sont utilisées comme entrée par aucun des deux services.

Source	Module	Fréquence de mise à jour
Fear & Greed Index	parametrage.py → API Alternative.me	Temps réel à chaque chargement Streamlit
Cours BTC 30j	app_benchmark.py → API CoinGecko	Temps réel à chaque chargement
OHLCV historiques	Bloc1 API → Bloc3 pipeline	Cron horaire + journalier

4.4 Documentation et procédures

Document	Contenu
Bloc2_veille/README.md	Installation (uv sync), lancement veille et benchmark Streamlit
Bloc3_ml/README.md	Architecture pipeline, config YAML, MLflow, API

	classification
docs/benchmark_services_ia.md	Benchmark formel C7 : expression de besoin, comparaison, conclusions
docs/methodologie_agile.md	Conduite de projet réelle (solo, sans board Kanban actif, backlog GitHub Issues)
litellm_config.yaml	Configuration du proxy LiteLLM (modèle, clé API source)
.env.example	Variables ANTHROPIC_API_KEY, LITELLM_MASTER_KEY/SALT_KEY/PROXY_URL/VIRTUAL_KEY documentées

5. Conclusion

Ce projet illustre un chemin d'intégration d'un service d'IA préexistant qui n'a pas été linéaire : un premier essai naïf (envoyer les données de marché à un LLM cloud) a fonctionné assez pour révéler ses propres limites (coût en tokens, non-reproductibilité, faible pertinence sur une tâche quantitative), ce qui a orienté une veille formalisée vers un modèle ML supervisé auto-hébergé, tout en conservant le LLM comme outil complémentaire d'analyse qualitative.

Le service finalement paramétré n'est donc pas le premier candidat testé, mais le résultat de deux itérations de remise en question, chacune documentée par une veille distincte et déclenchée par un constat concret plutôt que par une préférence a priori.

L'architecture retenue (XGBoost local pour la prédiction + Claude en POC complémentaire) répond aux contraintes de coût, reproductibilité et éco-responsabilité identifiées dans le benchmark, tout en laissant une porte ouverte à l'enrichissement de l'interface utilisateur par des synthèses en langage naturel.

Le paramétrage de C8 s'est aussi concrétisé par un proxy LiteLLM réellement déployé (§4.2.5), qui centralise le suivi de coût et de tokens et prépare l'ajout d'autres fournisseurs sans changer le code applicatif.

6. Sources consultées

- Alternative.me — Fear & Greed Index API (documentation, source primaire)
- Anthropic — Claude Platform Pricing et Model Deprecations (source primaire, éditeur)
- OpenAI — API Pricing GPT-4o (source primaire, éditeur)
- Mistral AI — Documentation et tarification (source primaire, éditeur)
- CoinGecko — API v3 market_chart (documentation, source primaire)
- LiteLLM — Spend Tracking, Virtual Keys, Simple Proxy / AI Gateway (docs.litellm.ai, source primaire, éditeur)
- Règlement spécifique RNCP37827 — Développeur en Intelligence Artificielle (Simplon, 2023)
- Référentiel Activités Compétences et évaluation — Bloc 2 (Simplon, 2023)
- Règlement UE MiCA — Markets in Crypto-Assets (cadre réglementaire crypto UE)
- Règlement UE AI Act — Intelligence artificielle (obligations transparence)
- Commission Nationale Informatique et Libertés (CNIL) — Guide RGPD
- Valentin Haüy — Recommandations accessibilité documents numériques
- Atalan AcceDe — Guide accessibilité des contenus web
- XGBoost Documentation — Parameters et Python API (source primaire)
- MLflow Documentation — Tracking et Model Registry (source primaire)
- Crisostomo, R. & Mykhalyuk, D. (2026) — Large Language Models and Stock Investing: Is the Human Factor Required? (arXiv:2603.19944, vérifié)
- Wang, S. et al. (2024) — StockTime: A Time Series Specialized Large Language Model Architecture for Stock Price Prediction (arXiv:2409.08281, vérifié)
- A Review of Large Language Models for Stock Price Forecasting from a Hedge-Fund Perspective (arXiv:2605.05211)
- Empowering Time Series Analysis with Large Language Models: A Survey (arXiv:2402.03182)
- Sources internes : docs/benchmark_services_ia.md, Bloc2_veille/, Bloc3_ml/, docs/methodologie_agile.md